

Cloud Computing Architecture

Semester project report

Group 54

Aaron Achermann - 20-940-805

Indro Born - 21-936-208

Matteo Pelossi – 22-952-444

Systems Group
Department of Computer Science
ETH Zurich
May 14, 2026

Instructions

- **Please do not modify the template!** Except for adding your solutions in the labeled places, and inputting your group number, names and legi-NR on the title page.

Divergence from the template can lead to subtraction of points on graded parts of the project.

- Parts 1 and 2 of the project are **ungraded**. They will help you analyze the behavior of the applications you will have to run on parts 3 and 4 (graded), for which you will have to design a scheduling policy **based on the information** you will gather on **parts 1 and 2**.
- Be conservative with your credits! Parts 3 and 4 might need extra credits for experimentation. Parts 1 and 2 should cost you approximately **15 USD**.
- **Use of AI Tools:** The use of AI tools must adhere to [ETH regulations](#). **You take responsibility for the content you submit.** You must disclose any use of GenAI and other AI tools in your work and avoid sharing copyrighted, private, or confidential information with commercial AI platforms. Failure to comply with these guidelines may result in disciplinary action.

Part 3 [68 points]

1. [34 points] Describe and analyze your hand-crafted policy.
 - (a) [17 points] With your scheduling policy, run the entire workflow **3 separate times**. For each run, measure the execution time of each batch job, as well as the latency outputs of memcached, running with a steady client load of 30K QPS. For each batch application, compute the mean and standard deviation of the execution time ¹ across three runs. Also, compute the mean and standard deviation of the total time to complete all jobs - the makespan of all jobs. Fill in the table below. Finally, compute the SLO violation ratio for memcached for the three runs; the number of data points with 95th percentile latency > 1ms, as a fraction of the total number of data points. The SLO violation ratio should be calculated during the time from when the first batch-job-container starts running to when the last batch-job-container stops running.

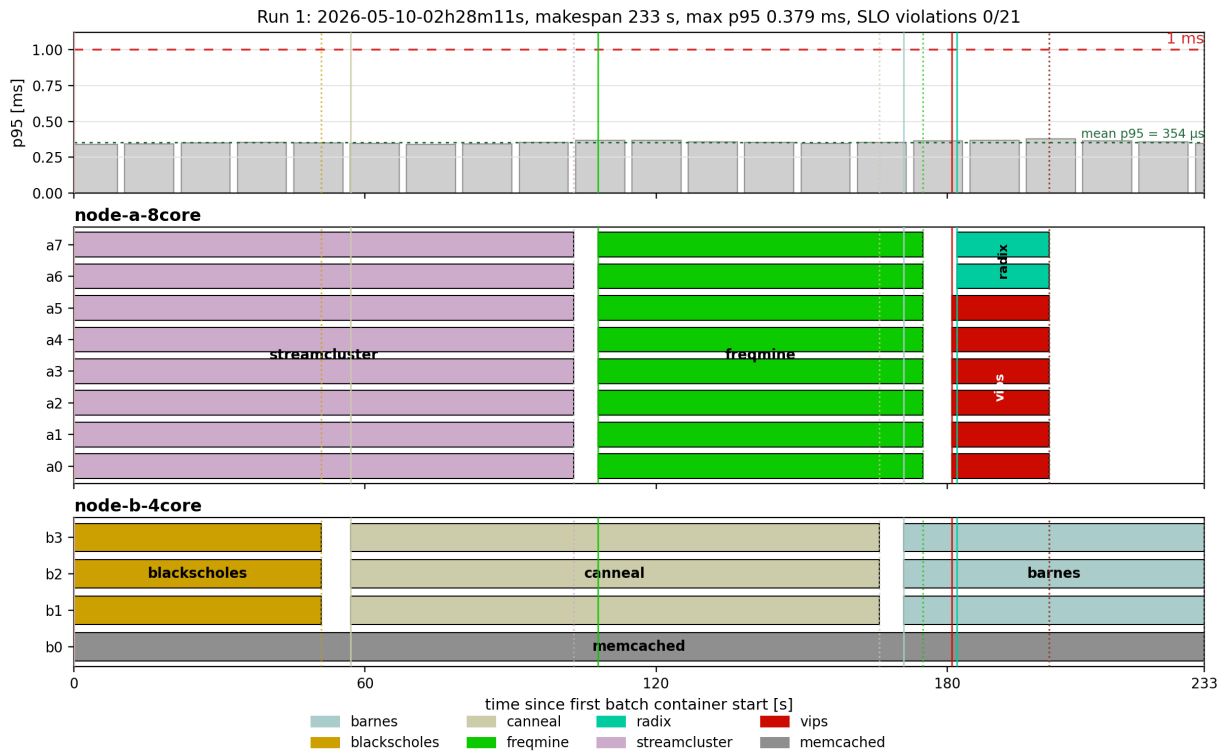
job name	mean time [s]	std [s]
barnes	63.33	1.53
blackscholes	47.67	3.06
canneal	107.33	1.53
freqmine	67.33	0.58
radix	20.67	2.08
streamcluster	113.33	16.20
vips	21.33	1.53
total time	232.33	3.06

Answer: The values in the table are computed from the container `startedAt` and `finishedAt` timestamps in `results.json`. In the augmented `mcperf` output, `ts_start` and `ts_end` are Unix epoch timestamps in milliseconds that bound the measurement interval represented by one p95 latency row. All three executions completed the seven batch jobs successfully. During the interval from the first batch container start to the last batch container finish, no measured memcached p95 sample exceeded the 1 ms SLO; the resulting SLO violation ratio is 0.000.

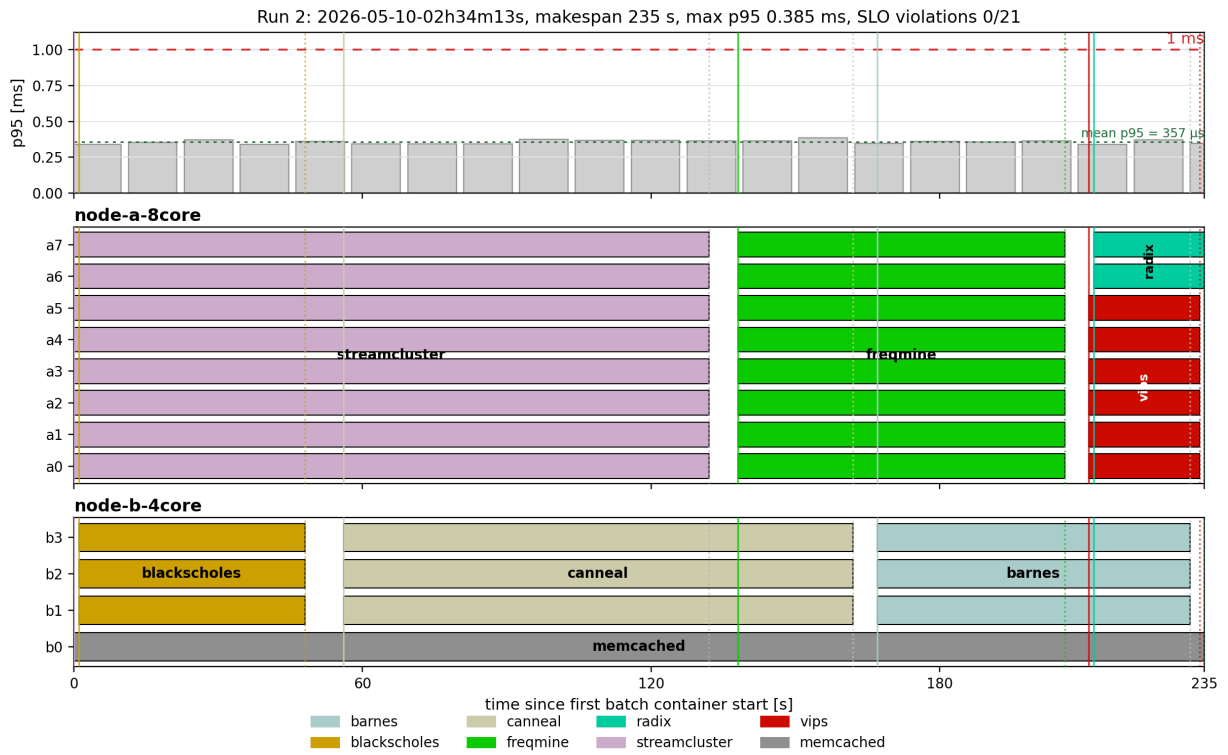
Create 3 bar plots (one for each run) of memcached p95 latency (y-axis) over time (x-axis), with annotations showing when each batch job started and ended, also indicating the machine each of them is running on. Using the augmented version of `mcperf`, you get two additional columns in the output: `ts_start` and `ts_end`. Use them to determine the width of each bar in the bar plot, while the height should represent the p95 latency. Align the x axis so that $x = 0$ coincides with the starting time of the first container. For each core in each machine show what job it is running using the colors proposed in this template (you can find them in `main.tex`). For example, draw a bar in **the color of vips** for core x from when vips is started to when it ends if vips ran on core x .

Plots:

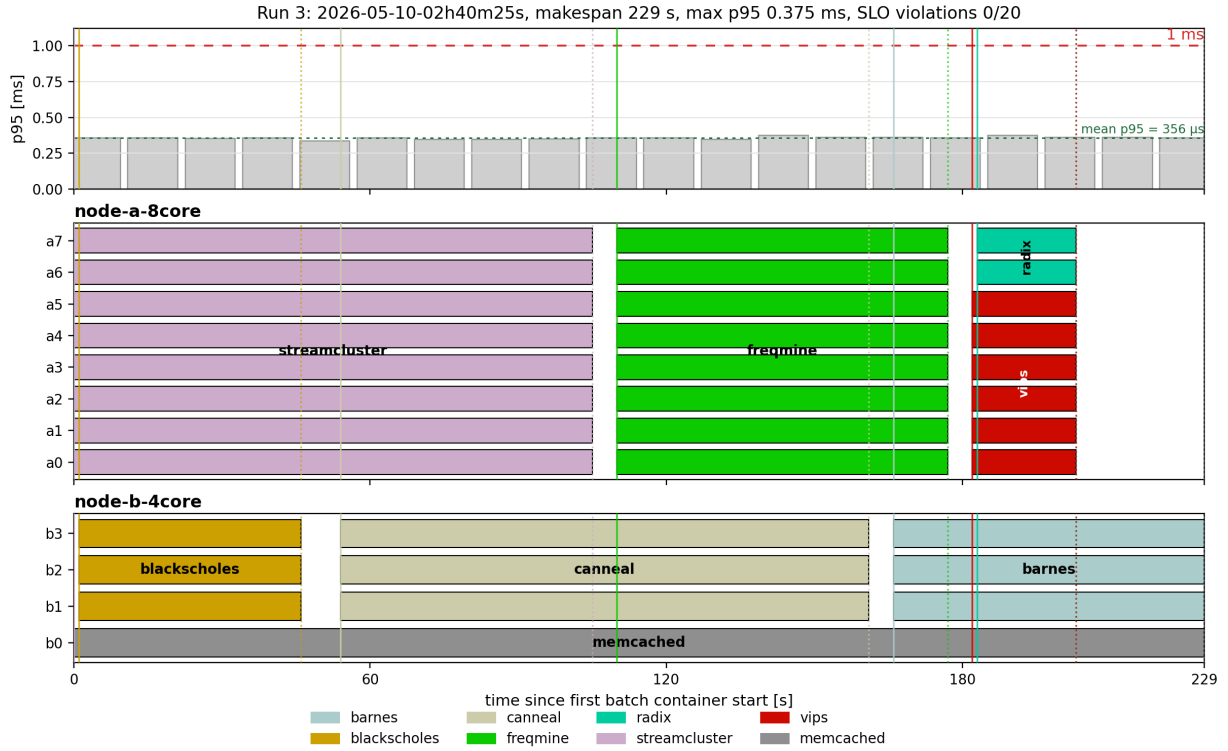
¹Here, you should only consider the runtime, excluding time spans during which the container is paused.



Run 1. Memcached p95 latency over time and per-core placement on **node-a-8core** and **node-b-4core**.
The red dashed line marks the 1 ms SLO.



Run 2. Memcached p95 latency over time and per-core placement on **node-a-8core** and **node-b-4core**.
The red dashed line marks the 1 ms SLO.



Run 3. Memcached p95 latency over time and per-core placement on **node-a-8core** and **node-b-4core**. The red dashed line marks the 1 ms SLO.

(b) [17 points] Describe and justify the “optimal” scheduling policy you have designed.

- Which node does memcached run on? Why?

Answer: Memcached runs on **node-b-4core**, pinned to core 0 with one server thread. The reasoning behind the placement of the memcached pod is multifaceted. The first constraint that immediately came to our attention was the physical shape of the two benchmark machines. By analyzing `part3.yaml`, we noticed that the two VMs request different resources and have different shapes: **node-a-8core** is an **e2-standard-8** machine, while **node-b-4core** is an **n2d-highcpu-4** machine. The **e2** shape has a less predictable CPU allocation: it can map to slower Intel CPUs or to a faster AMD Rome CPU. In our captured logs, however, the 8-core VM was consistently assigned AMD Rome. The second node, **node-b-4core**, was an **n2d** high-CPU machine and in our experiments was consistently assigned AMD Milan. The most important difference is therefore not only the number of cores, but also the memory density and CPU shape: **node-a-8core** has 8 cores and about 32 GB of memory, while **node-b-4core** has only 4 cores and about 4 GB of memory. Given this information, we tried to build the policy using only the knowledge extracted from Part 1 and Part 2. From Part 1, the immediate observation was that memcached was less susceptible to DRAM bandwidth interference than to CPU and cache interference. This made it reasonable to keep memcached on a single isolated core, while allowing the remaining three cores of the same machine to execute batch jobs sequentially. We did not want memcached to compete directly for a core, for L1d/L1i, or for L2 with a batch job. Pinning it to core 0 on the AMD Milan node

gave us a stronger and more predictable latency-critical core, and cores 1-3 remained available for the node-B job sequence. Across the three selected final-policy runs, this placement produced zero memcached SLO violations.

- Which node does each of the 7 batch jobs run on? Why?

Answer:

- **barnes**: **barnes** runs on **node-b-4core**, pinned to cores 1-3, with 3 threads. We place it at the end of the node-B chain, after **canneal**. Although **barnes** benefits from more cores, it is also cache-sensitive, especially to LLC pressure. At this point in the schedule, **node-a-8core** is already used by the final **vips+radix** overlap, while **node-b-4core** has three isolated batch cores available next to the memcached core. Running **barnes** there avoids adding a third job to node A and keeps the policy simple and reproducible.
- **blackscholes**: **blackscholes** runs on **node-b-4core**, pinned to cores 1-3, with 3 threads. This is one of the safest jobs to place next to memcached: from Part 2 it showed only moderate interference sensitivity, and from the speedup experiment it did not gain much from going from 4 to 8 threads. Therefore it is a good first job for the 3-core node-B batch region, while memcached remains isolated on core 0.
- **canneal**: **canneal** runs on **node-b-4core**, pinned to cores 1-3, with 3 threads, after **blackscholes**. **canneal** is memory and LLC sensitive, but it also has poor scalability compared with the jobs that benefit most from all 8 cores. Since the second machine has only three batch cores once memcached is allocated, we use it for jobs where the opportunity cost of not using 8 cores is lower. The measured runs confirmed that this placement stayed within the memcached SLO.
- **freqmine**: **freqmine** runs on **node-a-8core**, pinned to cores 0-7, with 8 threads, after **streamcluster**. From Part 2, **freqmine** has a significant speedup at high thread counts, especially from 4 to 8 threads. It also has irregular control flow and cache pressure, so we preferred not to colocate it with memcached. Giving it the full 8-core node reduces the critical path without risking direct interference on memcached's private caches.
- **radix**: **radix** runs on **node-a-8core**, pinned to cores 6-7, with 2 threads, after **freqmine**. During experimentation we discovered that **radix** was a highly unpredictable and unstable job. We therefore never schedule it on **node-b-4core**, because it can crash the 4-core, 4 GB VM. We also restrict its thread count to a power-of-two value from {1, 2, 4, 8}, because non-power-of-two thread counts can make it crash. The final schedule uses 2 threads, which satisfies this constraint and lets **radix** share the tail of **node-a-8core** with **vips** on disjoint cores.
- **streamcluster**: **streamcluster** runs on **node-a-8core**, pinned to cores 0-7, with 8 threads, and starts immediately. This job showed the strongest scalability in Part 2, so the 8-core node is the natural place for it. Starting it first also makes sense because it is one of the longest jobs in the workflow; delaying it would directly increase the makespan.
- **vips**: **vips** runs on **node-a-8core**, pinned to cores 0-5, with 6 threads, after **freqmine**. **vips** benefits from parallelism but has diminishing returns at high thread counts, so six cores are enough to make it short while leaving cores 6-7 for **radix**. This is the only intentional batch-job colocation in the final tail of

the schedule, and the two jobs use disjoint core sets.

- Which jobs run concurrently / are colocated? Why?

Answer: We use “concurrent” to mean that two jobs overlap in time, and “colocated” to mean that they overlap on the same VM. Concurrency across different VMs is mainly a makespan choice: it does not create direct CPU-cache contention between the two batch jobs, but it keeps both paid benchmark nodes busy. Colocation is the more delicate decision, because colocated jobs can compete for cores, caches, memory bandwidth, and memory capacity.

With this distinction, the final policy runs the node-A and node-B work streams at the same time only to avoid leaving a machine idle. On `node-a-8core`, `streamcluster` runs first on the full 8-core machine, followed by `frequimine` on the full 8-core machine. On `node-b-4core`, `blackscholes`, `canneal`, and `barnes` run sequentially on cores 1-3, while `memcached` stays isolated on core 0.

The only intentional same-node batch colocation is at the tail: `vips` and `radix` both start after `frequimine` on `node-a-8core`. `vips` uses cores 0-5, while `radix` uses cores 6-7. We chose this colocation because both jobs are short tail jobs and, in our empirical experimentation, running them one after the other gave little end-to-end benefit once the roughly 6 s container startup overhead was included. Keeping `radix` on node A also avoids the instability we observed when trying to run it on the 4-core, 4 GB node. Thus, the policy allows overlap when it reduces idle time, but avoids same-core competition and uses same-node colocation only once, with disjoint core sets.

- In which order did you run the 7 batch jobs? Why?

Order: `streamcluster` and `blackscholes` start first. Then `frequimine` starts after `streamcluster`, while `canneal` starts after `blackscholes`. Finally, `vips` and `radix` start together after `frequimine`, and `barnes` starts after `canneal`.

Why: The order is chosen to keep both machines busy while respecting the different strengths of the two nodes. The higher core count of `node-a-8core` is used first for the jobs with the clearest 8-thread benefit: `streamcluster` and then `frequimine`. On the other side, after allocating one core to `memcached`, `node-b-4core` only has three batch cores available, so it runs a sequential chain of 3-thread jobs: `blackscholes`, `canneal`, and `barnes`. The tail of the schedule is then compressed by running `vips` and `radix` concurrently on disjoint node-A cores. This ordering was a direct consequence of the Part 2 speedup results and of the practical observation that container startup time becomes important for short jobs.

- How many threads have you used for each of the 7 batch jobs? Why?

Answer:

- `barnes`: 3 threads, pinned to cores 1-3 on `node-b-4core`. This matches the available batch cores on node B while keeping `memcached` isolated on core 0.
- `blackscholes`: 3 threads, pinned to cores 1-3 on `node-b-4core`. This job does not gain enough from 8 threads to justify using node A, so three threads are the natural choice on the `memcached` node.
- `canneal`: 3 threads, pinned to cores 1-3 on `node-b-4core`. Its scalability is limited by memory/cache behavior, so we avoid spending the full 8-core machine

on it in the final policy.

- **freqmine**: 8 threads, pinned to cores 0-7 on **node-a-8core**. The Part 2 speedup experiment showed that it benefits significantly from the jump to 8 threads.
- **radix**: 2 threads, pinned to cores 6-7 on **node-a-8core**. This choice is not only a performance choice, but also a stability constraint: **radix** must use one of {1, 2, 4, 8} threads and should not be scheduled on **node-b-4core**. Two threads are enough for the tail overlap with **vips**, while staying inside the safe power-of-two thread set.
- **streamcluster**: 8 threads, pinned to cores 0-7 on **node-a-8core**. This was the strongest scaling workload in Part 2, so it receives the full 8-core node.
- **vips**: 6 threads, pinned to cores 0-5 on **node-a-8core**. Six threads preserve most of the parallel benefit while leaving two disjoint cores for **radix**.

- Which files did you modify or add and in what way? Which Kubernetes features did you use?

Answer: We modified **part3.yaml** to define the final cluster shape, including the two benchmark nodes **node-a-8core** and **node-b-4core**, their machine types, and their identifying Kubernetes node labels. We also added bootstrap user-data for the client machines so that the augmented **mcperf** tooling and the load-agent services were installed and started automatically. The scheduling policy itself was represented in YAML policy files and then rendered into per-run Kubernetes manifests. The Kubernetes features we used were deliberately simple: node labels and **nodeSelector** fields to force each pod onto the intended VM, Kubernetes Jobs/Pods to run each PARSEC/SPLASH-2x batch container, and explicit CPU affinity inside the container command via **taskset**. The policy controller launched jobs according to the **after** dependencies in the YAML file, which gave us a small sequential DAG over the seven batch jobs.

- Describe the design choices, ideas and trade-offs you took into account while creating your scheduler (if not already mentioned above). Describe how your policy (and its performance) compares to another policy that you experimented with (in particular to the second-best policy that you designed).

Answer: The most important design choice was to keep the policy simple and constrained. We decided not to explore simultaneous multithreading or hyperthreading policies. This means that we did not test policies where multiple jobs compete on the same core, and we did not use thread counts larger than the number of pinned cores. This reduced the search space and also matched the lessons from Part 1 and Part 2: the dangerous interference sources for memcached were CPU and private/shared cache effects, especially L1d, L1i, L2, and LLC misses. Therefore, the scheduler avoids direct core sharing and uses disjoint core sets whenever jobs overlap.

Another trade-off was between using all 8 cores for every scalable job and keeping the overall makespan short. Some jobs, such as **streamcluster** and **freqmine**, clearly deserved the full 8-core node because their 4-to-8-thread speedup was significant and their runtime was long enough to affect the critical path. Other jobs, especially short tail jobs, were affected by container startup time. We measured this overhead to be about 6s when the image was already cached. This changed the decision for the tail of the schedule: running **vips** and **radix** concurrently on disjoint cores was

better than launching them sequentially with larger thread counts.

Compared with earlier explored policies that used more aggressive or less balanced overlap, the final hand-crafted policy is more conservative and more robust. It gives up some possible parallelism, but it keeps memcached on a dedicated core, avoids overcommitting cores, keeps `radix` away from the unstable node-B placement, and consistently produces zero SLO violations in the selected three runs. Its measured makespan over the three selected executions was 232.33s on average, with all seven batch jobs completing successfully. This is why we selected it as the final hand-crafted policy: it is not only fast enough, but also stable and easy to justify from the hardware and interference observations.

2. [34 points] Describe and analyze the AI-generated policy. [TODO: Xiaozhe]

(a) [17 points] **Report the performance:**

- Report the mean time of each batch job, as well as the makespan of all jobs for the AI-generated policy. Also, report the SLO violation ratio for memcached for the AI-generated policy. *Note: If the LLM failed to produce a valid solution after 3+ attempts, provide the error logs and your best-performing manual tweak for partial credit.*

job name	mean time [s] (Baseline)	mean time [s] (AI)
barnes		
blackscholes		
canneal		
freqmine		
radix		
streamcluster		
vips		
total time		

- Create 3 bar plots (one for each run), for the final AI-generated policy, of memcached p95 latency (y-axis) over time (x-axis), with annotations showing when each batch job started and ended, also indicating the machine each of them is running on. Use the same method as described in the previous question to create the bar plots.

Answer:

- Create two line plots (one for the p95 latency and one for SLO violations) showing policy performance over iterations.

Answer:

(b) [17 points] **Analysis of the Evolutionary Process:** Describe the process of how you used OpenEvolve to design the AI-generated policy. For example, you can include the following details in your answer (You are not limited to these questions. You can include any other details you find relevant and interesting). Your response will be graded based on the thoroughness of the exploration and clarity in explanation:

- How many iterations did you run through OpenEvolve with the LLMs to get to the final policy? Which LLM(s) did you use?

- How do you measure success? For example, how do you design the ‘combined_score’ or the success metric, given that there are two objectives (makespan and SLO violations)?
- Describe the initial policy you fed to the LLM(s) and why you chose this as the initial policy.
- Describe how the policy changes during the process, and how it impacted the performance of the policy (e.g., tail latency; SLO violations).
- Explain why did this AI-generated policy perform better (or worse) than the baseline?
- Identify one “hallucination” or logical flaw the model produced during the process and how you identified it.

Answer:

Please attach your modified/added YAML files, run scripts, OpenEvolve scripts, experiment outputs and the report as a zip file. You can find more detailed instructions about the submission in the project description file.

Important: The search space of all possible policies is exponential and you do not have enough credits to run all of them. We do not ask you to find the policy that minimizes the total running time, but rather to design a policy that has a reasonable running time, does not violate the SLO, and takes into account the characteristics of the first two parts of the project.